

```
// File: frontend/src/main.js
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import 'highlight.js/styles/github.css'
import zhCn from 'element-plus/dist/locale/zh-cn.mjs'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'

import App from './App.vue'
import router from './router'
import './assets/styles/main.scss'

const app = createApp(App)

// 注册 Element Plus 图标
for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}

app.use(createPinia())
app.use(router)
app.use(ElementPlus, { locale: zhCn })

app.mount('#app')
```

```
// File: frontend/src/router/index.js
import { createRouter, createWebHistory } from 'vue-router'
import { AUTH_ENABLED } from '@config/app'

const routes = [
  {
    path: '/login',
    name: 'Login',
    component: () => import('@views/Login.vue'),
    meta: { guest: true, title: '登录' },
  },
  {
    path: '/',
    component: () => import('@views/Layout.vue'),
    children: [
      { path: '', name: 'Home', component: () => import('@views/Dashboard.vue'), meta: { title: '首页' } },
      { path: 'research-data', name: 'ResearchData', component: () => import('@views/ResearchData.vue'), meta: { title: '研究数据' } },
      { path: 'parks', name: 'Parks', component: () => import('@views/Parks.vue'), meta: { title: '遗址公园' } },
      { path: 'parks/:id', name: 'ParkDetail', component: () => import('@views/ParkDetail.vue'), meta: { title: '公园详情' } },
      { path: 'map', name: 'Map', component: () => import('@views/MapView.vue'), meta: { title: '地图浏览', flush: true, hideFooter: true } },
    ],
  },
]
```

```

    { path: 'compare', name: 'Comparison', component: () => import('@/views/Comparison.vue'), meta:
{ title: '对比分析' } },
    { path: 'comparison', redirect: '/compare' },
    { path: 'knowledge-graph', name: 'KnowledgeGraph', component: () =>
import('@/views/KnowledgeGraph.vue'), meta: { title: '知识图谱', wide: true } },
    { path: 'assistant', name: 'Assistant', component: () => import('@/views/Chat.vue'), meta: { title:
'AI 助手', wide: true, hideFooter: true } },
    { path: 'chat', redirect: '/assistant' },
    { path: 'library', name: 'Library', component: () => import('@/views/KnowledgeBase.vue'), meta:
{ title: '知识库' } },
    { path: 'knowledge-base', redirect: '/library' },
    { path: 'bibliometrics', name: 'Bibliometrics', component: () =>
import('@/views/Bibliometrics.vue'), meta: { title: '文献计量' } },
    { path: 'about', name: 'About', component: () => import('@/views/About.vue'), meta: { title: '关于我
们' } },
    { path: 'data-management', name: 'DataManagement', component: () =>
import('@/views/admin/DataManage.vue'), meta: { title: '数据管理', admin: true } },
    { path: 'admin', redirect: '/data-management' },
  ],
},
{ path: '/*', redirect: '/' },
]

```

```
const router = createRouter({ history: createWebHistory(), routes, scrollBehavior: () => ({ top: 0 }) })
```

```

router.beforeEach((to) => {
  document.title = `${to.meta.title || '研究平台'} - 遗址公园研究平台`
  if (!AUTH_ENABLED) return true
  if (to.meta.guest) return true
  if (to.meta.admin && !localStorage.getItem('token')) {
    return { path: '/login', query: { redirect: to.fullPath } }
  }
  return true
})

```

```
export default router
```

```
// File: backend/app/api/chat.py
```

```

import json
import uuid
from datetime import datetime, timezone
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from sse_starlette.sse import EventSourceResponse

from ..core.database import get_db, get_neo4j, get_chroma_collection
from ..core.auth import get_current_user
from ..models.user import User
from ..schemas.chat import ChatRequest, ChatResponse, ConversationMeta, ConversationListResponse
from ..services.rag_engine import RAGEngine

```

```
router = APIRouter()

# 存储会话（内存），包含元数据
conversation_store: dict = {}
MAX_CONVERSATIONS = 100

def _get_or_create_conversation(conversation_id: str, first_message: str = "") -> str:
    """获取或创建会话，返回 conversation_id"""
    if not conversation_id:
        conversation_id = str(uuid.uuid4())

    if conversation_id not in conversation_store:
        if len(conversation_store) >= MAX_CONVERSATIONS:
            oldest_key = next(iter(conversation_store))
            del conversation_store[oldest_key]
        conversation_store[conversation_id] = {
            "id": conversation_id,
            "title": first_message[:30] if first_message else "新对话",
            "created_at": datetime.now(timezone.utc).isoformat(),
            "last_active": datetime.now(timezone.utc).isoformat(),
            "messages": [],
        }
    else:
        conversation_store[conversation_id]["last_active"] = datetime.now(timezone.utc).isoformat()

    return conversation_id

@router.post("", response_model=ChatResponse)
async def chat(
    request: ChatRequest,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """聊天接口（非流式）"""
    conversation_id = _get_or_create_conversation(request.conversation_id, request.message)
    conv = conversation_store[conversation_id]
    history = conv["messages"]

    neo4j_session = get_neo4j()
    rag = RAGEngine(db=db, neo4j_session=neo4j_session, chroma=get_chroma_collection())

    result = await rag.query(
        question=request.message,
        conversation_history=history,
    )

    history.append({"role": "user", "content": request.message})
    history.append({"role": "assistant", "content": result["answer"]})
```

```

    if len(history) > 20:
        conv["messages"] = history[-20:]

    conv["title"] = conv["title"] or request.message[:30]
    conv["last_active"] = datetime.now(timezone.utc).isoformat()

    return ChatResponse(
        message=result["answer"],
        conversation_id=conversation_id,
        sources=result.get("sources", []),
        chart_data=result.get("chart_data"),
    )

@router.post("/stream")
async def chat_stream(
    request: ChatRequest,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """流式聊天接口 (SSE)"""
    conversation_id = _get_or_create_conversation(request.conversation_id, request.message)
    conv = conversation_store[conversation_id]
    history = conv["messages"]

    neo4j_session = get_neo4j()
    rag = RAGEngine(db=db, neo4j_session=neo4j_session, chroma=get_chroma_collection())

    async def event_generator():
        full_answer = ""
        try:
            async for event in rag.stream_query(
                question=request.message,
                conversation_history=history,
            ):
                event_type = event["type"]
                event_data = event["data"]

                if event_type == "token":
                    full_answer += event_data
                    yield {"event": "token", "data": json.dumps({"token": event_data},
ensure_ascii=False)}

                elif event_type == "chart_data":
                    yield {"event": "chart_data", "data": json.dumps(event_data, ensure_ascii=False)}
                elif event_type == "sources":
                    yield {"event": "sources", "data": json.dumps(event_data, ensure_ascii=False)}
                elif event_type == "done":
                    yield {"event": "done", "data": "{}"}

        # 保存对话历史
        history.append({"role": "user", "content": request.message})

```

```
        history.append({"role": "assistant", "content": full_answer})

    if len(history) > 20:
        conv["messages"] = history[-20:]

    conv["last_active"] = datetime.now(timezone.utc).isoformat()

except Exception as e:
    yield {"event": "error", "data": json.dumps({"detail": str(e)}, ensure_ascii=False)}

return EventSourceResponse(
    event_generator(),
    headers={
        "Cache-Control": "no-cache",
        "X-Accel-Buffering": "no",
        "Connection": "keep-alive",
    },
)

@router.get("/conversations", response_model=ConversationListResponse)
async def list_conversations(
    current_user: User = Depends(get_current_user),
):
    """获取所有会话列表"""
    convs = []
    for cid, cdata in conversation_store.items():
        convs.append(ConversationMeta(
            id=cdata["id"],
            title=cdata["title"],
            message_count=len(cdata["messages"]),
            created_at=cdata["created_at"],
            last_active=cdata["last_active"],
        ))
    convs.sort(key=lambda x: x.last_active, reverse=True)
    return ConversationListResponse(conversations=convs)

@router.get("/history/{conversation_id}")
async def get_history(
    conversation_id: str,
    current_user: User = Depends(get_current_user),
):
    """获取会话历史"""
    conv = conversation_store.get(conversation_id)
    if not conv:
        return []
    return conv["messages"]

@router.delete("/conversation/{conversation_id}")
```

```
async def delete_conversation(
    conversation_id: str,
    current_user: User = Depends(get_current_user),
):
    """删除会话"""
    if conversation_id in conversation_store:
        del conversation_store[conversation_id]
    return {"message": "对话已删除"}

@router.delete("/history/{conversation_id}")
async def clear_history(
    conversation_id: str,
    current_user: User = Depends(get_current_user),
):
    """清空会话历史（兼容旧接口）"""
    if conversation_id in conversation_store:
        del conversation_store[conversation_id]
    return {"message": "对话历史已清除"}

// File: backend/app/api/kg.py
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session
from typing import Optional

from ..core.database import get_db, get_neo4j, neo4j_driver
from ..core.auth import get_current_user
from ..models.user import User
from ..models.park import Park
from ..models.score import Score
from ..models.indicator import Indicator

router = APIRouter()

def has_neo4j():
    """检查 Neo4j 是否可用"""
    return neo4j_driver is not None

@router.get("/stats")
async def kg_stats(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """知识图谱统计信息"""
    if has_neo4j():
        # Neo4j 模式
        with get_neo4j() as session:
            node_count = session.run("MATCH (n) RETURN count(n) as c").single()["c"]
            rel_count = session.run("MATCH ()-[r]->() RETURN count(r) as c").single()["c"]
```

```

        return {
            "total_nodes": node_count,
            "total_relations": rel_count,
            "source": "neo4j",
        }
    else:
        # SQLite 模式 - 从关系数据库生成统计
        parks_count = db.query(Park).count()
        indicators_count = db.query(Indicator).count()
        scores_count = db.query(Score).count()

        return {
            "total_nodes": parks_count + indicators_count,
            "total_relations": scores_count,
            "source": "sqlite",
            "node_types": [
                {"labels": ["Park"], "count": parks_count},
                {"labels": ["Indicator"], "count": indicators_count},
            ],
            "relation_types": [
                {"type": "HAS_SCORE", "count": scores_count},
            ],
        }

@router.get("/entity/{entity_type}")
async def get_entities(
    entity_type: str,
    name: Optional[str] = Query(None),
    limit: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取实体列表"""
    if has_neo4j():
        with get_neo4j() as session:
            if name:
                result = session.run(
                    f"MATCH (n:{entity_type}) WHERE n.name CONTAINS $name RETURN n LIMIT $limit",
                    name=name, limit=limit
                )
            else:
                result = session.run(f"MATCH (n:{entity_type}) RETURN n LIMIT $limit", limit=limit)
            return [dict(record["n"]) for record in result]

    # SQLite 模式
    if entity_type == "Park":
        query = db.query(Park)
        if name:
            query = query.filter(Park.name.contains(name) | Park.short_name.contains(name))
        parks = query.limit(limit).all()

```

```

        return [
            {
                "name": p.name,
                "short_name": p.short_name,
                "type": p.park_type,
                "province": p.province,
                "city": p.city,
                "batch": p.batch,
                "area": p.total_area,
            }
            for p in parks
        ]

    elif entity_type == "Indicator":
        indicators = db.query(Indicator).limit(limit).all()
        return [
            {
                "code": i.code,
                "name": i.name,
                "dimension": i.dimension,
            }
            for i in indicators
        ]

    return []

@router.get("/entity/{entity_type}/{name}")
async def get_entity_detail(
    entity_type: str,
    name: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取实体详情"""
    if has_neo4j():
        with get_neo4j() as session:
            result = session.run(
                f"MATCH (n:{entity_type} {{name: $name}}) OPTIONAL MATCH (n)-[r]->(m) RETURN n,
collect({{relation: type(r), target: m.name}}) as relations",
                name=name
            )
            record = result.single()
            if record:
                return {"entity": dict(record["n"]), "relations": record["relations"]}
            return {"error": "实体不存在"}

# SQLite 模式
if entity_type == "Park":
    park = db.query(Park).filter(
        (Park.name == name) | (Park.short_name == name)
    )

```

```

    ).first()
    if park:
        scores = (
            db.query(Score, Indicator)
            .join(Indicator, Score.indicator_id == Indicator.id)
            .filter(Score.park_id == park.id)
            .all()
        )
    return {
        "entity": {
            "name": park.name,
            "short_name": park.short_name,
            "type": park.park_type,
            "province": park.province,
        },
        "relations": [
            {
                "relation": "HAS_SCORE",
                "target": f"{ind.code} {ind.name}",
                "value": sc.normalized_score,
            }
            for sc, ind in scores
        ],
    }

    return {"error": "实体不存在"}

@router.get("/neighbors/{name}")
async def get_neighbors(
    name: str,
    depth: int = Query(2, ge=1, le=3),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取邻居节点"""
    if has_neo4j():
        with get_neo4j() as session:
            result = session.run(
                f"MATCH path = (n {{name: $name}})-[*1..{depth}]->(m) RETURN path LIMIT 100",
                name=name
            )
            nodes = {}
            edges = []
            for record in result:
                path = record["path"]
                for node in path.nodes:
                    nid = node.element_id
                    if nid not in nodes:
                        nodes[nid] = dict(node)
                for rel in path.relationships:

```

```

        edges.append({
            "source": rel.start_node.get("name"),
            "target": rel.end_node.get("name"),
            "type": rel.type,
        })
    return {"nodes": list(nodes.values()), "edges": edges}

# SQLite 模式 - 返回公园及其评分关系
park = db.query(Park).filter(
    (Park.name == name) | (Park.short_name == name)
).first()

if not park:
    # 返回所有公园的概览
    parks = db.query(Park).all()
    nodes = [{"name": p.short_name or p.name, "type": p.park_type, "labels": ["Park"]} for p in parks]
    edges = []

    # 同类型公园之间建立关系
    type_groups = {}
    for p in parks:
        if p.park_type not in type_groups:
            type_groups[p.park_type] = []
        type_groups[p.park_type].append(p.short_name or p.name)

    for ptype, names in type_groups.items():
        for i in range(len(names)):
            for j in range(i + 1, len(names)):
                edges.append({"source": names[i], "target": names[j], "type": "SIMILAR_TYPE"})

    return {"nodes": nodes, "edges": edges}

# 返回特定公园的详情
scores = (
    db.query(Score, Indicator)
    .join(Indicator, Score.indicator_id == Indicator.id)
    .filter(Score.park_id == park.id)
    .all()
)

nodes = [{"name": park.short_name or park.name, "type": park.park_type, "labels": ["Park"]}
edges = []

for sc, ind in scores:
    nodes.append({
        "name": f"{ind.code} {ind.name}",
        "type": "Indicator",
        "labels": ["Indicator"],
        "score": sc.normalized_score,
    })
    edges.append({

```

```

        "source": park.short_name or park.name,
        "target": f"{ind.code} {ind.name}",
        "type": "HAS_SCORE",
        "value": sc.normalized_score,
    })

    return {"nodes": nodes, "edges": edges}

@router.get("/search")
async def search_kg(
    q: str = Query(...),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """搜索知识图谱"""
    results = []

    # 搜索公园
    parks = db.query(Park).filter(
        Park.name.contains(q) | Park.short_name.contains(q)
    ).all()
    for p in parks:
        results.append({"labels": ["Park"], "data": {"name": p.name, "short_name": p.short_name, "type":
p.park_type}})

    # 搜索指标
    indicators = db.query(Indicator).filter(
        Indicator.name.contains(q) | Indicator.code.contains(q)
    ).all()
    for ind in indicators:
        results.append({"labels": ["Indicator"], "data": {"code": ind.code, "name": ind.name, "dimension":
ind.dimension}})

    return results

// File: backend/app/api/knowledge.py
"""知识库 API - 含 ChromaDB 索引、语义搜索、GB/T 7714 导出、健康检查、综述生成"""
import os
import json
import asyncio
from typing import List
from fastapi import APIRouter, Depends, HTTPException, UploadFile, File, Query
from sqlalchemy.orm import Session

from ..core.database import get_db
from ..core.auth import get_current_user, get_admin_user
from ..models.user import User
from ..models.literature import Literature, Citation, LitRelation
from ..services.kb_processor import extract_metadata, discover_relations
from ..services.kb_indexer import index_literature, delete_index, rebuild_index, semantic_search

```

```

from ..services.kb_lint import lint_knowledge_base, generate_literature_review

router = APIRouter()

# 存储路径
KB_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))), "knowledge_base")

def _ensure_kb_dirs():
    os.makedirs(os.path.join(KB_DIR, "raw"), exist_ok=True)
    os.makedirs(os.path.join(KB_DIR, "wiki"), exist_ok=True)

def _get_rel_target_title(db, rel, lit_id: int) -> str:
    """获取关系目标文献的标题，安全处理不存在的情况"""
    target_id = rel.target_id if rel.source_id == lit_id else rel.source_id
    if not target_id:
        return "未知"
    target = db.query(Literature).filter(Literature.id == target_id).first()
    return target.title if target else "未知"

# ===== 上传文献 =====

@router.post("/upload")
async def upload_literature(
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """上传文献并自动提取元数据 + 向量索引"""
    _ensure_kb_dirs()

    ext = file.filename.lower().rsplit('.', 1)[-1] if '.' in file.filename else ''
    if ext not in ('pdf', 'docx', 'doc', 'txt', 'md'):
        raise HTTPException(400, "仅支持 .pdf / .docx / .doc / .txt / .md")

    content = await file.read()
    if len(content) > 100 * 1024 * 1024:
        raise HTTPException(400, "文件不能超过 100MB")

    # 提取文本
    from ..services.doc_parser import extract_text
    text = extract_text(content, file.filename)
    if not text or len(text.strip()) < 50:
        raise HTTPException(400, "文件内容为空或无法识别")

    # 保存原文
    safe_name = file.filename.replace(" ", "_")
    raw_path = os.path.join(KB_DIR, "raw", safe_name)
    with open(raw_path, "wb") as f:

```

```
f.write(content)

# LLM 提取元数据（线程池避免阻塞）
loop = asyncio.get_running_loop()
try:
    metadata = await asyncio.wait_for(
        loop.run_in_executor(None, extract_metadata, text),
        timeout=180.0,
    )
except asyncio.TimeoutError:
    metadata = {
        "title": file.filename, "year": 2024, "doc_type": "其他",
        "authors": [], "journal": "", "keywords": [], "abstract": text[:500],
        "funding": "", "methodology": "", "core_argument": "",
        "references": [], "parks_mentioned": [], "concepts_mentioned": [],
    }

# 保存数据库
lit = Literature(
    title=metadata.get("title", file.filename),
    authors=json.dumps(metadata.get("authors", []), ensure_ascii=False),
    year=metadata.get("year", 2024),
    journal=metadata.get("journal", ""),
    keywords=json.dumps(metadata.get("keywords", []), ensure_ascii=False),
    abstract=metadata.get("abstract", ""),
    doc_type=metadata.get("doc_type", "其他"),
    funding=metadata.get("funding", ""),
    methodology=metadata.get("methodology", ""),
    core_argument=metadata.get("core_argument", ""),
    file_path=raw_path,
    wiki_path="",
    raw_metadata=json.dumps(metadata, ensure_ascii=False),
)
db.add(lit)
db.flush()

# 保存引文关系
refs = metadata.get("references", [])
for ref in refs:
    if isinstance(ref, dict):
        existing = db.query(Literature).filter(
            Literature.title == ref.get("title", "")
        ).first()
        citation = Citation(
            source_id=lit.id,
            target_title=ref.get("title", ""),
            target_author=ref.get("author", ""),
            target_year=ref.get("year", ""),
            target_id=existing.id if existing else None,
        )
        db.add(citation)
```

```

db.commit()
db.refresh(lit)

# 向量索引
kws = metadata.get("keywords", [])
index_literature(lit.id, lit.title, lit.abstract or "", kws)

# 异步发现与已有文献的关系
try:
    existing = db.query(Literature).filter(Literature.id != lit.id).all()
    existing_data = [
        {"id": e.id, "title": e.title, "year": e.year, "abstract": (e.abstract or "")[:200]}
        for e in existing[:20]
    ]
    new_data = {"id": lit.id, "title": lit.title, "year": lit.year, "abstract": lit.abstract or ""}
    relations = await asyncio.wait_for(
        loop.run_in_executor(None, discover_relations, new_data, existing_data),
        timeout=120.0,
    )

    for rel in relations:
        target_title = rel.get("target_title", "")
        if not target_title:
            continue
        target = db.query(Literature).filter(Literature.title == target_title).first()
        if target:
            db.add(LitRelation(
                source_id=lit.id,
                target_id=target.id,
                relation_type=rel.get("type", "shares_topic"),
                strength=rel.get("strength", 0.5),
                explanation=rel.get("explanation", ""),
            ))
    db.commit()
except (asyncio.TimeoutError, Exception) as e:
    print(f"[KB] 关系发现跳过: {str(e)[:80]}")

return {
    "id": lit.id,
    "title": lit.title,
    "year": lit.year,
    "keywords": metadata.get("keywords", []),
    "abstract": (lit.abstract or "")[:200],
    "citations_found": len(refs),
}

# ===== 文献列表 =====

@router.get("/list")

```

```

async def list_literature(
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    keyword: str = Query(None),
    year_from: int = Query(None),
    year_to: int = Query(None),
    doc_type: str = Query(None),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """列出文献"""
    query = db.query(Literature)
    if keyword:
        like = f"%{keyword}%"
        query = query.filter(
            Literature.title.like(like) |
            Literature.abstract.like(like) |
            Literature.keywords.like(like)
        )
    if year_from:
        query = query.filter(Literature.year >= year_from)
    if year_to:
        query = query.filter(Literature.year <= year_to)
    if doc_type:
        query = query.filter(Literature.doc_type == doc_type)

    total = query.count()
    items = query.order_by(Literature.created_at.desc()).offset(
        (page - 1) * page_size
    ).limit(page_size).all()

    return {
        "total": total,
        "page": page,
        "items": [
            {
                "id": l.id,
                "title": l.title,
                "authors": json.loads(l.authors) if l.authors else [],
                "year": l.year,
                "journal": l.journal,
                "keywords": json.loads(l.keywords) if l.keywords else [],
                "abstract": (l.abstract or "")[:300],
                "doc_type": l.doc_type,
                "citation_count": l.citation_count,
                "created_at": str(l.created_at),
            }
            for l in items
        ],
    }

```

```

# ===== 文献详情 =====

@router.get("/detail/{lit_id}")
async def get_literature_detail(
    lit_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """文献详情 + 关联文献"""
    lit = db.query(Literature).filter(Literature.id == lit_id).first()
    if not lit:
        raise HTTPException(404, "文献不存在")

    citations_from = db.query(Citation).filter(Citation.source_id == lit_id).all()
    citations_to = db.query(Citation).filter(Citation.target_id == lit_id).all()

    relations = db.query(LitRelation).filter(
        (LitRelation.source_id == lit_id) | (LitRelation.target_id == lit_id)
    ).all()

    kws = json.loads(lit.keywords) if lit.keywords else []
    related = []
    if kws:
        conditions = [Literature.keywords.like(f"%{kw}%") for kw in kws[:5]]
        from sqlalchemy import or_
        related = db.query(Literature).filter(
            Literature.id != lit_id,
            or_(*conditions),
        ).limit(10).all()

    return {
        "id": lit.id,
        "title": lit.title,
        "authors": json.loads(lit.authors) if lit.authors else [],
        "year": lit.year,
        "journal": lit.journal,
        "keywords": kws,
        "abstract": lit.abstract,
        "doc_type": lit.doc_type,
        "funding": lit.funding,
        "methodology": lit.methodology,
        "core_argument": lit.core_argument,
        "citation_count": lit.citation_count,
        "created_at": str(lit.created_at),
        "citations_from": [
            {"title": c.target_title, "author": c.target_author, "year": c.target_year, "in_db":
bool(c.target_id)}
            for c in citations_from
        ],
        "cited_by": [

```

```

        {"title": c.source.title, "year": c.source.year} if c.source else {"title": "未知"}
        for c in citations_to
    ],
    "relations": [
        {
            "target_title": _get_rel_target_title(db, r, lit_id),
            "type": r.relation_type,
            "explanation": r.explanation,
        }
        for r in relations
    ],
    "related": [
        {"id": r.id, "title": r.title, "year": r.year}
        for r in related
    ],
}

```

===== 删除文献 =====

```

@router.delete("/delete/{lit_id}")
async def delete_literature(
    lit_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """删除文献"""
    lit = db.query(Literature).filter(Literature.id == lit_id).first()
    if not lit:
        raise HTTPException(404, "文献不存在")
    if lit.file_path and os.path.exists(lit.file_path):
        os.remove(lit.file_path)
    db.delete(lit)
    db.commit()
    # 删除向量索引
    delete_index(lit_id)
    return {"message": "已删除"}

```

===== 文献计量 =====

```

@router.get("/bibliometrics")
async def get_bibliometrics(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取全库文献计量指标"""
    from ..services.bibliometrics import compute_all
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(None, compute_all, db)

```

```
# ===== 语义搜索 (ChromaDB 优先, SQL 兜底) =====
```

```
@router.get("/search")
async def search_literature(
    q: str = Query(...),
    limit: int = Query(10, ge=1, le=50),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """语义搜索 (ChromaDB 向量检索优先, SQL LIKE 兜底)"""
    results = semantic_search(q, limit, db)
    return {"query": q, "results": results, "mode": "semantic" if any(r.get("similarity") for r in
results) else "sql"}
```

```
# ===== 向量索引管理 =====
```

```
@router.post("/reindex")
async def reindex_knowledge_base(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """全量重建文献向量索引"""
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(None, rebuild_index, db)
    return result
```

```
# ===== GB/T 7714 引用格式导出 =====
```

```
@router.get("/cite/{lit_id}")
async def get_citation_gbt7714(
    lit_id: int,
    style: str = Query("gbt7714", description="引用格式: gbt7714 | apa | mla"),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取 GB/T 7714 / APA / MLA 格式引用"""
    lit = db.query(Literature).filter(Literature.id == lit_id).first()
    if not lit:
        raise HTTPException(404, "文献不存在")

    authors = json.loads(lit.authors) if lit.authors else []
    formatted = _format_citation(lit, authors, style)
    return {"lit_id": lit_id, "style": style, "citation": formatted}
```

```
@router.post("/cite/batch")
async def batch_citation(
    lit_ids: List[int],
```

```

style: str = Query("gbt7714"),
db: Session = Depends(get_db),
current_user: User = Depends(get_current_user),
):
    """批量生成引用"""
    lits = db.query(Literature).filter(Literature.id.in_(lit_ids)).all()
    id_map = {l.id: l for l in lits}
    results = []
    for lid in lit_ids:
        lit = id_map.get(lid)
        if lit:
            authors = json.loads(lit.authors) if lit.authors else []
            results.append({
                "lit_id": lid,
                "title": lit.title,
                "citation": _format_citation(lit, authors, style),
            })
    return {"style": style, "citations": results}

def _format_citation(lit, authors: list, style: str) -> str:
    """格式化引用"""
    author_str = ""
    if authors:
        names = [a.get("name", "") for a in authors[:3] if isinstance(a, dict)]
        if names:
            author_str = ", ".join(names)
            if len(authors) > 3:
                author_str += ", 等" if style == "gbt7714" else " et al."

    year = str(lit.year) if lit.year else ""
    title = lit.title or ""

    if style == "gbt7714":
        # GB/T 7714-2015
        parts = []
        if author_str:
            parts.append(author_str + ".")
        parts.append(f"{title}[{'lit.doc_type' or 'J'}].")
        if lit.journal:
            parts.append(f"{lit.journal}.")
        if year:
            parts.append(f"{year}.")
        return " ".join(parts)

    elif style == "apa":
        parts = []
        if author_str:
            parts.append(f"{author_str} ({year}).")
        else:
            parts.append(f"({year}).")

```

```

        parts.append(f"{title}.")
        if lit.journal:
            parts.append(f"*{lit.journal}*.")
        return " ".join(parts)

    elif style == "mla":
        parts = []
        if author_str:
            parts.append(f'{author_str}. "{title}."')
        else:
            parts.append(f'"{title}."')
        if lit.journal:
            parts.append(f"*{lit.journal}*")
        if year:
            parts.append(f"{year}.")
        return " ".join(parts)

    return f"{author_str}. {title}. {year}."

# ===== 知识库图谱数据 =====

@router.get("/graph")
async def get_knowledge_graph(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取文献知识图谱数据（引用网络 + 语义关系 + 关键词关联）"""
    lits = db.query(Literature).all()
    citations = db.query(Citation).all()
    relations = db.query(LitRelation).all()

    nodes = []
    edges = []
    lit_ids = {l.id for l in lits}

    # 文献节点
    for l in lits:
        kws = json.loads(l.keywords) if l.keywords else []
        authors = json.loads(l.authors) if l.authors else []
        author_names = [a.get("name", "") for a in authors[:2] if isinstance(a, dict)]
        nodes.append({
            "id": f"lit_{l.id}",
            "label": l.title[:30],
            "type": "literature",
            "year": l.year,
            "doc_type": l.doc_type,
            "keywords": kws[:5],
            "authors": author_names,
            "symbolSize": 20 + len(kws) * 2,
        })

```

```

# 关键词节点 (高频)
kw_counter = {}
for l in lits:
    kws = json.loads(l.keywords) if l.keywords else []
    for kw in kws:
        if kw not in kw_counter:
            kw_counter[kw] = {"count": 0, "lits": []}
        kw_counter[kw]["count"] += 1
        kw_counter[kw]["lits"].append(l.id)

top_kws = sorted(kw_counter.items(), key=lambda x: x[1]["count"], reverse=True)[:30]
for kw, info in top_kws:
    nodes.append({
        "id": f"kw_{kw}",
        "label": kw,
        "type": "keyword",
        "count": info["count"],
        "symbolSize": 10 + info["count"] * 3,
    })
# 关键词 → 文献边
for lit_id in info["lits"]:
    edges.append({
        "source": f"kw_{kw}",
        "target": f"lit_{lit_id}",
        "type": "has_keyword",
    })

# 引用边
for c in citations:
    if c.source_id in lit_ids:
        edges.append({
            "source": f"lit_{c.source_id}",
            "target": f"lit_{c.target_id}" if c.target_id and c.target_id in lit_ids else
f"lit_{c.source_id}",
            "type": "cites",
            "label": f"[{c.target_year or '?'}] {c.target_title[:20]}",
        })

# 语义关系边
for r in relations:
    if r.source_id in lit_ids and r.target_id in lit_ids:
        edges.append({
            "source": f"lit_{r.source_id}",
            "target": f"lit_{r.target_id}",
            "type": r.relation_type,
            "label": r.relation_type,
        })

return {"nodes": nodes, "edges": edges, "total": len(nodes)}

```

```
# ===== 知识库健康检查 =====

@router.get("/lint")
async def lint_kb(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """知识库健康检查"""
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(None, lint_knowledge_base, db)

# ===== 自动综述 =====

@router.get("/review")
async def get_literature_review(
    topic: str = Query("", description="综述主题，为空则全库综述"),
    max_lits: int = Query(20, ge=5, le=50),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """生成文献综述结构化数据"""
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(None, generate_literature_review, db, topic, max_lits)
    return result

@router.post("/review/generate")
async def generate_review_with_llm(
    topic: str = Query("", description="综述主题"),
    max_lits: int = Query(20, ge=5, le=50),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_admin_user),
):
    """使用 LLM 生成完整文献综述"""
    from ..core.config import settings

    if not settings.ai_available:
        raise HTTPException(400, "需要配置 AI API key 才能生成综述")

    # 先获取结构化数据
    data = generate_literature_review(db, topic, max_lits)
    if "error" in data:
        return data

    # 拼接 LLM 提示词
    lit_texts = []
    for lit in data["lits"]:
        authors = ", ".join([a.get("name", "") for a in lit["authors"][:3]])
        lit_texts.append(
```

```

        f"- [{lit['year']}] {authors}. {lit['title']}. "
        f"论点: {lit.get('core_argument', '')}. 方法: {lit.get('methodology', '')}"
    )

    top_kws = ", ".join(data.get("top_keywords", [])[:10])
    methods = ", ".join([m["name"] for m in data.get("methods", [])[:5]])

    prompt = f"""你是文化遗产保护研究领域的资深学者。请根据以下文献数据，撰写一篇学术综述。

## 综述主题
{topic or "文化遗产保护研究"}

## 文献概况
- 相关文献: {data['total_found']} 篇
- 时间跨度: {data.get('years_range', [])}
- 高频关键词: {top_kws}
- 常用方法: {methods}

## 文献列表
{chr(10).join(lit_texts)}

## 要求
1. 按研究脉络分段: 研究缘起 → 主要议题 → 方法论演进 → 前沿趋势 → 研究展望
2. 使用学术语言，标注引用（如 [1]、[2]）
3. 指出研究空白和方法论局限
4. 1500-2500 字
5. 使用 Markdown 格式"""

    try:
        from openai import OpenAI
        client = OpenAI(
            api_key=settings.DASHSCOPE_API_KEY,
            base_url=settings.LLM_BASE_URL or "https://dashscope.aliyuncs.com/compatible-mode/v1",
            timeout=120.0,
        )
        response = client.chat.completions.create(
            model=settings.AI_MODEL,
            messages=[
                {"role": "system", "content": "你是文化遗产保护研究领域的资深学者，擅长撰写文献综述。"},
                {"role": "user", "content": prompt},
            ],
            max_tokens=4096,
            temperature=0.7,
        )
        review_text = response.choices[0].message.content

    return {
        "topic": topic or "文化遗产保护",
        "lits_referenced": data["total_found"],
        "review": review_text,
        "references": [

```

```
        {"id": lit["id"], "title": lit["title"], "year": lit["year"]}
        for lit in data["lits"]
    ],
}
except Exception as e:
    raise HTTPException(500, f"综述生成失败: {str(e)}")

// File: backend/app/api/parks.py
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
from typing import Optional, List

from ..core.database import get_db
from ..core.auth import get_current_user
from ..models.park import Park
from ..models.score import Score
from ..models.indicator import Indicator
from ..schemas.park import ParkResponse, ParkListResponse, ScoreInfo

router = APIRouter()

def build_park_response(park, db):
    """构建公园响应数据"""
    scores = (
        db.query(Score, Indicator)
        .join(Indicator, Score.indicator_id == Indicator.id)
        .filter(Score.park_id == park.id)
        .all()
    )
    score_list = [
        ScoreInfo(
            indicator_code=ind.code,
            indicator_name=ind.name,
            dimension=ind.dimension,
            score=sc.normalized_score,
            grade=sc.grade,
        )
        for sc, ind in scores
    ]

    return ParkResponse(
        id=park.id,
        name=park.name,
        short_name=park.short_name,
        park_type=park.park_type,
        batch=park.batch,
        province=park.province,
        city=park.city,
        district=park.district,
        longitude=park.longitude,
```

```
        latitude=park.latitude,
        total_area=park.total_area,
        core_area=park.core_area,
        world_heritage=park.world_heritage or 0,
        aaa_level=park.aaa_level,
        open_year=park.open_year,
        description=park.description,
        scores=score_list,
    )

@router.get("", response_model=ParkListResponse)
async def list_parks(
    park_type: Optional[str] = Query(None),
    province: Optional[str] = Query(None),
    batch: Optional[int] = Query(None),
    keyword: Optional[str] = Query(None),
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
):
    query = db.query(Park)

    if park_type:
        query = query.filter(Park.park_type == park_type)
    if province:
        query = query.filter(Park.province == province)
    if batch:
        query = query.filter(Park.batch == batch)
    if keyword:
        query = query.filter(
            Park.name.contains(keyword) | Park.short_name.contains(keyword)
        )

    total = query.count()
    parks = query.offset((page - 1) * page_size).limit(page_size).all()

    result = [build_park_response(park, db) for park in parks]

    return ParkListResponse(total=total, items=result)

@router.get("/{park_id}", response_model=ParkResponse)
async def get_park(park_id: int, db: Session = Depends(get_db)):
    park = db.query(Park).filter(Park.id == park_id).first()
    if not park:
        raise HTTPException(status_code=404, detail="遗址公园不存在")
    return build_park_response(park, db)

@router.get("/{park_id}/sites")
```

```

async def get_park_sites(park_id: int, db: Session = Depends(get_db)):
    from ..models.site import Site
    park = db.query(Park).filter(Park.id == park_id).first()
    if not park:
        raise HTTPException(status_code=404, detail="遗址公园不存在")
    sites = db.query(Site).filter(Site.park_id == park_id).all()
    return [
        {
            "id": s.id, "site_name": s.site_name, "site_type": s.site_type,
            "period": s.period, "integrity_score": s.integrity_score,
            "safety_score": s.safety_score, "description": s.description,
        }
        for s in sites
    ]

```

```

@router.get("/{park_id}/scores")
async def get_park_scores(park_id: int, db: Session = Depends(get_db)):
    park = db.query(Park).filter(Park.id == park_id).first()
    if not park:
        raise HTTPException(status_code=404, detail="遗址公园不存在")

    scores = (
        db.query(Score, Indicator)
        .join(Indicator, Score.indicator_id == Indicator.id)
        .filter(Score.park_id == park_id)
        .order_by(Indicator.code)
        .all()
    )
    return [
        {
            "code": ind.code,
            "name": ind.name,
            "dimension": ind.dimension,
            "sub_dimension": ind.sub_dimension,
            "score": sc.normalized_score,
            "grade": sc.grade,
            "evidence": sc.evidence,
        }
        for sc, ind in scores
    ]

```

```

// File: backend/app/api/statistics.py
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session
from sqlalchemy import func

from ..core.database import get_db
from ..core.auth import get_current_user
from ..models.park import Park
from ..models.site import Site

```

```
from ..models.score import Score
from ..models.indicator import Indicator
from ..models.user import User

router = APIRouter()

@router.get("/overview")
async def overview(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """数据概览"""
    total_parks = db.query(Park).count()
    total_sites = db.query(Site).count()
    total_indicators = db.query(Indicator).count()

    # 按类型统计
    type_stats = (
        db.query(Park.park_type, func.count(Park.id))
        .group_by(Park.park_type)
        .all()
    )

    # 按省份统计
    province_stats = (
        db.query(Park.province, func.count(Park.id))
        .group_by(Park.province)
        .all()
    )

    # 按批次统计
    batch_stats = (
        db.query(Park.batch, func.count(Park.id))
        .filter(Park.batch.isnot(None))
        .group_by(Park.batch)
        .all()
    )

    return {
        "total_parks": total_parks,
        "total_sites": total_sites,
        "total_indicators": total_indicators,
        "by_type": {t: c for t, c in type_stats},
        "by_province": {p: c for p, c in province_stats},
        "by_batch": {b: c for b, c in batch_stats},
    }

@router.get("/comparison")
async def comparison(
```

```

    park_ids: str = Query(..., description="公园 ID 列表, 逗号分隔"),
    dimension: str = Query(None, description="评估维度"),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """多公园对比"""
    ids = [int(x.strip()) for x in park_ids.split(",")]
    if len(ids) > 10:
        ids = ids[:10]

    parks = db.query(Park).filter(Park.id.in_(ids)).all()

    result = []
    for park in parks:
        query = (
            db.query(Score, Indicator)
            .join(Indicator, Score.indicator_id == Indicator.id)
            .filter(Score.park_id == park.id)
        )
        if dimension:
            query = query.filter(Indicator.dimension == dimension)

        scores = query.all()

        score_data = {}
        for sc, ind in scores:
            score_data[ind.code] = {
                "name": ind.name,
                "dimension": ind.dimension,
                "score": sc.normalized_score,
                "grade": sc.grade,
            }

        result.append({
            "park_id": park.id,
            "park_name": park.short_name or park.name,
            "park_type": park.park_type,
            "scores": score_data,
        })

    return result

@router.get("/dimension")
async def dimension_scores(
    park_type: str = Query(None, description="公园类型"),
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """各维度得分统计"""
    query = (

```

```

        db.query(
            Park.park_type,
            Indicator.dimension,
            func.avg(Score.normalized_score).label("avg_score"),
            func.count(Score.id).label("count"),
        )
        .join(Score, Park.id == Score.park_id)
        .join(Indicator, Score.indicator_id == Indicator.id)
    )

    if park_type:
        query = query.filter(Park.park_type == park_type)

    results = query.group_by(Park.park_type, Indicator.dimension).all()

    return [
        {
            "park_type": r.park_type,
            "dimension": r.dimension,
            "avg_score": round(float(r.avg_score), 2),
            "count": r.count,
        }
        for r in results
    ]

@router.get("/radar/{park_id}")
async def radar_data(
    park_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    """获取单个公园的雷达图数据"""
    park = db.query(Park).filter(Park.id == park_id).first()
    if not park:
        return {"error": "公园不存在"}

    scores = (
        db.query(Indicator.dimension, func.avg(Score.normalized_score).label("avg"))
        .join(Score, Indicator.id == Score.indicator_id)
        .filter(Score.park_id == park_id)
        .group_by(Indicator.dimension)
        .all()
    )

    return {
        "park_name": park.short_name or park.name,
        "dimensions": [
            {"dimension": r.dimension, "score": round(float(r.avg), 2)}
            for r in scores
        ],
    },

```

```

    }

// File: backend/app/services/rag_engine.py
"""
RAG 问答引擎
- 保留本地模板模式作为降级方案
- 当配置了真实 AI key 时, 调用 LLM API 生成回答
- 支持流式 (SSE) 输出
- 集成知识库文献检索
"""

import json
from typing import List, Dict, Any, Optional, AsyncGenerator

from ..core.config import settings
from .llm_provider import get_llm_provider, MockProvider
from .prompt_builder import build_messages

class RAGEngine:
    """基于知识图谱的 RAG 问答引擎"""

    def __init__(self, db, neo4j_session=None, chroma=None):
        self.db = db
        self.neo4j = neo4j_session
        self.chroma = chroma
        self.llm = get_llm_provider()

    async def query(
        self,
        question: str,
        conversation_history: List[Dict] = None,
    ) -> Dict[str, Any]:
        """处理用户查询"""
        # 1-4: RAG 管线 (与本地模式共用)
        intent = self._classify_intent(question)
        db_context = self._query_database(question, intent)
        doc_context = self._query_vector_store(question) if self.chroma else ""
        lit_context = self._query_knowledge_base(question)
        context = self._assemble_context(db_context, doc_context, lit_context)
        chart_data = self._generate_chart_data(question, intent)
        if results and results["documents"]:
            return "\n".join([f"相关文档: {doc[:300]}" for doc in results["documents"][0]])
        except Exception:
            pass
        return ""

    def _query_knowledge_base(self, question: str) -> str:
        """
        查询文献知识库
        判断用户问题是否涉及学术/遗产保护理论, 若是则检索文献
        同时拉取每篇文献的引用和关联关系
        """

```

